HTML & CSS • MODULE 01

HTML Basics & Structure

Build Your First Web Page From Scratch

HTML FOUNDATIONS

LEARNING OUTCOME

By the end of this module you will write a complete, valid HTML5 document from memory, understand the DOM tree structure, use all essential HTML elements correctly, and know the difference between block and inline elements.

Skills You Will Gain

- Write correct HTML5 document boilerplate from memory
- Use headings, paragraphs, links, images, and lists confidently
- Distinguish block elements from inline elements
- Add SEO meta tags and viewport meta for responsiveness
- Create tables for tabular data with thead, tbody, tfoot
- Understand how browsers parse and render HTML

■ HTML is the skeleton of every website on Earth. Every web developer — frontend, backend, full-stack — must know HTML. It takes one day to learn the basics and a career to master the nuances.

SECTION 1

What is HTML?

HTML (HyperText Markup Language) is the standard language for creating web pages. It uses **elements** represented by **tags** to describe the structure and meaning of web content. Browsers parse HTML and render it as visual pages.

HTML is NOT a programming language

HTML is a MARKUP language — it describes STRUCTURE and MEANING, not logic. It has no variables, no loops, no conditions. The three layers of a web page: HTML — Structure (the skeleton) CSS — Presentation (the skin and style) JavaScript — Behaviour (the muscles and brain)

SECTION 2

The HTML5 Document Structure

```
<!DOCTYPE html> <html lang="en"> <head> <!-- METADATA: not visible on the page --> <meta charset="UTF-8"> <meta name="viewport" content="width=device-width, initial-scale=1.0"> <meta name="description" content="Page description for search engines (SEO)"> <title>Page Title – shown in browser tab</title> <link rel="stylesheet" href="style.css"> <link rel="icon" href="favicon.ico"> </head> <body> <!-- CONTENT: everything visible on the page --> <header> <h1>Site Name</h1> <nav> <a href="/about">About</a> <a href="/contact">Contact</a> </nav> </header> <main> <h2>Main Heading</h2> <p>A paragraph of text with <strong>bold</strong> and <em>italic</em>.</p> <img src="photo.jpg" alt="Description of the image" width="800" height="600"> <a href="https://example.com" target="_blank" rel="noopener">External Link</a> </main> <footer> <p>&copy; 2024 My Website</p> </footer> <script src="app.js"></script> </body> </html>
```

SECTION 3

Essential HTML Elements

Element	Purpose & Notes
h1 to h6	Headings — h1 is most important, use ONE h1 per page, h2-h6 for sub-headings
p	Paragraph — block of text, auto spacing above and below
a href='url'	Hyperlink — add target='_blank' for new tab, always add rel='noopener'
img src='' alt=''	Image — ALWAYS add alt text for accessibility and SEO, add width and height to prevent layout shift
ul / ol / li	Unordered (bullet) / Ordered (numbered) lists — li must be direct child of ul or ol
strong / em	Bold (semantic importance) / Italic (emphasis) — screen readers treat these differently from b and i
span	Inline container — use for targeting specific text with CSS or JavaScript
div	Block container — generic grouping with no semantic meaning, use for layout

br	Line break — use sparingly, prefer CSS margin/padding for spacing
hr	Horizontal rule — thematic break between content sections

SECTION 4

Block vs Inline Elements

The Critical Difference

BLOCK elements: take full available width, start on a new line, stack vertically Examples: div, p, h1-h6, ul, ol, li, table, header, main, section, article, footer INLINE elements: only as wide as their content, flow with surrounding text, sit side by side Examples: span, a, strong, em, img, input, button, label, code Rule: Block elements can contain inline elements, but inline elements should NOT wrap block elements. WRONG: ... RIGHT: ...

SECTION 5

Lists and Tables

```
<!-- Unordered list --> <ul> <li>HTML</li> <li>CSS</li> <li>JavaScript</li> </ul> <!-- Ordered list with nested list --> <ol> <li>Frontend <ul> <li>HTML &amp; CSS</li> <li>JavaScript</li> </ul> </li> <li>Backend</li> </ol> <!-- Description list (for key-value pairs, glossaries) --> <dl> <dt>HTML</dt> <dd>HyperText Markup Language — structure of web pages</dd> <dt>CSS</dt> <dd>Cascading Style Sheets — presentation and layout</dd> </dl> <!-- Table with semantic structure --> <table> <caption>Monthly Sales Data</caption> <thead> <tr> <th scope="col">Month</th> <th scope="col">Revenue</th> <th scope="col">Growth</th> </tr> </thead> <tbody> <tr> <td>January</td> <td>■1,20,000</td> <td>+12%</td> </tr> </tbody> <tfoot> <tr> <td colspan="2">Total</td> <td>■14,40,000</td> </tr> </tfoot> </table>
```

SECTION 6

HTML Entities and Special Characters

Entity Code	Character	When to Use
<	<	Display a less-than sign (not parsed as HTML tag)
>	>	Display a greater-than sign
&	&	Display an ampersand (use in text, URLs)
©	©	Copyright symbol in footer
™	™	Trademark symbol
** **		Non-breaking space — prevents line break between two words
—	—	Em dash — for parenthetical phrases
€	€	Euro currency symbol

■ PRACTICE Q & A

Q1. What is the purpose of <!DOCTYPE html> at the top of every HTML file?

A: *It declares the document type to the browser — HTML5 in this case. Without it, browsers enter 'quirks mode' and apply old, inconsistent rendering rules. Always include it as the very first line.*

Q2. What does the viewport meta tag do and why is it essential?

A: *<meta name='viewport' content='width=device-width, initial-scale=1.0'> tells mobile browsers not to zoom out to fit the desktop-sized page. Without it, mobile browsers show a tiny, unreadable version of your desktop layout.*

Q3. What is the difference between strong and b, and between em and i?

A: *strong/em are semantic — they convey meaning (important, emphasised) and screen readers interpret them. b/i are purely visual — bold and italic with no semantic meaning. Always prefer strong and em.*

Q4. Why must every img tag have an alt attribute?

A: *Accessibility: screen readers read alt text aloud for visually impaired users. SEO: search engines index alt text. Fallback: displays when image fails to load. Leave alt='' for decorative images.*

Q5. Can a div be placed inside a span? Why or why not?

A: *No — div is a block element; span is inline. Placing a block element inside an inline element violates the HTML content model and causes unpredictable rendering. Always nest block inside block, inline inside inline or block.*

■ HTML Basics Cheat Sheet	
`<!DOCTYPE html>`	Required — declares HTML5
`<html lang='en'>`	Root element with language
`<meta charset='UTF-8'>`	Character encoding — first in head
`<meta name='viewport'>`	Mobile responsive — essential
`<title>`	Browser tab + SEO page title
`<link rel='stylesheet'>`	Link external CSS file
`h1 to h6`	Headings — one h1 per page
`<a href='url'>`	Hyperlink — always descriptive text
`<img src='' alt=''>`	Image — alt text always required
`&amp; &lt; &gt;`	HTML special character entities
`<strong> / <em>`	Bold / italic with semantic meaning
`<div> / <span>`	Block / inline generic containers